Smallstep CA Security Standard

Offline-root separation, constrained online issuance, private listener hardening, and evidence-driven operations

Version 1.0 | August 28, 2026

Canonical source digest: 72b9786e63256da2476c

This component installs and initializes a single-node Smallstep `step-ca` service for internal X.509 issuance on Rocky Linux. It is a reusable security standard, not a deployment record. CA names, DNS suffixes, addresses, fingerprints, provisioners, issuance policy, credentials, certificates, keys, and production evidence belong in the deployment's private system of record.

The baseline deliberately separates the offline root from the online issuer:

- the root private key is created and retained offline or in an approved HSM;
- the online host receives only the public root certificate, a signed intermediate certificate, and its encrypted private key;
- `setup` cannot accept, create, or install a root private key;
- the intermediate unlock password is a root-only file loaded through a systemd credential, never an environment variable or `ca.json` field;
- issuance is limited by an explicit X.509 policy and encrypted JWK provisioners;
- the CA listens only on one RFC 1918 address and an unprivileged port;
- systemd and the independently managed network firewall restrict client source networks.

This is a single-server availability model. Embedded Badger storage is not a high-availability database. A production design needing multiple active CA instances must use a supported shared database, synchronized configuration, and a separately reviewed load-balancing/failure model.

Reviewed compatibility

Component	Reviewed baseline
Operating system	Rocky Linux 9 or 10, x86_64
Step CLI	0.30.6 stable release
step-ca	0.30.2 stable release
systemd	RHEL-family systemd with credentials and IP address policy
Online state	Embedded Badger v2, single active server
TLS	TLS 1.2 through 1.3; renegotiation disabled

`packages.manifest` pins the official GitHub release RPM names and SHA-256 digests. The installer downloads exact immutable release assets and refuses a different installed version. Re-review the upstream release notes, release signatures, and test matrix before advancing either package.

Public/private boundary

The repository contains no usable CA material. Deployment-local inputs are:

- public root certificate;
- intermediate certificate signed by that root;
- encrypted intermediate private key;
- intermediate unlock password file;
- JWK provisioner JSON with encrypted private material;
- scoped X.509 issuance-policy JSON;
- server DNS name, private listener address, and allowed client networks.

Keep those values out of Git, shell history, tickets, and public validation logs. A private operations record should retain certificate fingerprints, serials, validity periods, approved policy exceptions, backup evidence, firewall path tests, and the exact immutable configuration revision promoted.

Offline preparation gate

Prepare the root and intermediate material on an offline ceremony host or through an approved HSM/KMS workflow. Do not run a convenience initialization that leaves `root_ca_key` on the online server. Before promotion, independently review these facts:

1. The root certificate is a CA certificate and its private key remains offline with at least two separately protected recovery copies.
2. The intermediate certificate is a CA certificate, is signed by the root, and has a constrained path length appropriate to the hierarchy.
3. The intermediate private key is encrypted with a strong unique password.
4. Provisioner passwords differ from the intermediate-key password.
5. The X.509 policy allows only the required internal DNS and IP namespaces; wildcard names are disabled unless separately approved.
6. The CA service identity certificate covers its exact private DNS name.
7. The root certificate has a documented client distribution and rotation plan before the first leaf certificate is issued.

The setup accepts a provisioner JSON array. This initial standard supports only encrypted `JWK` provisioners. Each object must contain `type: "JWK"`, a nonempty name, a public `key` object, and a nonempty `encryptedKey`. Plaintext password fields are rejected.

The policy input is the value of `authority.policy`, not a whole `ca.json`. For example, this non-deployable documentation fixture scopes issuance to reserved example names and addresses:

```
{
  "x509": {
    "allow": {
      "dns": ["*.internal.example.invalid"],
      "ip": ["10.20.0.0/16"]
    },
    "allowWildcardNames": false
  }
}
```

The policy file must not contain an SSH policy. SSH certificates, ACME, OIDC, SCEP, remote provisioner management, cloud KMS, and HSM-backed online signing are valuable capabilities but need their own threat model and are not silently enabled by this baseline.

Certificate lifetime policy

The default service-certificate lifetime is 24 hours. The preferred maximum is 720 hours (30 days), matching Smallstep's guidance that host and service certificates live for one month or less. Automated renewal should run well before expiry and alert on repeated failure. The configurable authority default may not exceed 168 hours; longer service lifetimes must be requested explicitly and remain bounded by the provisioner and authority maximum.

`setup` permits an explicitly selected maximum up to 840 hours (35 days) for a reviewed operational exception, such as a renewal threshold that needs a small scheduling margin. It prints an exception notice above 720 hours. The absolute ceiling is not a recommendation. A one-year leaf certificate is outside this standard because it lengthens credential exposure, delays policy/key rollover, and turns renewal automation into an infrequently exercised recovery event.

If immediate revocation is a requirement, short lifetimes alone are insufficient. Design and test CRL or other revocation behavior separately; this baseline leaves the experimental built-in CRL endpoint disabled.

Install

Review a non-mutating platform and package plan:

```
smallstep-ca/install --plan
```

Then install the exact packages and public verifier policy:

```
sudo smallstep-ca/install
```

Installation does not initialize a PKI or start `step-ca.service`. It fails if a package change would be required while an existing CA service is active.

Initialize the online CA

First review a plan. Paths and identifiers below are documentation fixtures; replace them only in the private deployment procedure:

```
smallstep-ca/setup --plan \
    --server-name ca.internal.example.invalid \
    --bind-address 10.20.30.40 \
    --port 8443 \
    --root-certificate-file /root/ca-import/root_ca.crt \
    --intermediate-certificate-file /root/ca-import/intermediate_ca.crt \
    --intermediate-private-key-file /root/ca-import/intermediate_ca_key \
    --intermediate-password-file /root/ca-import/intermediate.password \
    --provisioners-file /root/ca-import/provisioners.json \
    --x509-policy-file /root/ca-import/x509-policy.json \
    --allowed-client-network 10.20.0.0/16 \
    --network-controls-confirmed
```

Plan mode validates argument shape but intentionally reads no input file or secret. Before apply, make each private source file root-owned, non-symbolic, single-linked, and mode `0400` or `0600`; public certificates may be `0644`.

Apply only after private DNS resolves to the one listener address and the host, cloud, overlay, and upstream firewalls have independent allowed-source and denied-source evidence:

```
sudo smallstep-ca/setup \
    --server-name ca.internal.example.invalid \
    --bind-address 10.20.30.40 \
    --port 8443 \
    --root-certificate-file /root/ca-import/root_ca.crt \
    --intermediate-certificate-file /root/ca-import/intermediate_ca.crt \
    --intermediate-private-key-file /root/ca-import/intermediate_ca_key \
    --intermediate-password-file /root/ca-import/intermediate.password \
    --provisioners-file /root/ca-import/provisioners.json \
    --x509-policy-file /root/ca-import/x509-policy.json \
    --allowed-client-network 10.20.0.0/16 \
    --network-controls-confirmed
```

Apply is first-install-only. It validates the certificate chain and matching encrypted intermediate key, creates the unprivileged `step` identity, installs root-owned material, renders the configuration and systemd policy, starts the service, and runs the strict verifier. On failure it disables the new service and removes only the targets created by that failed first-install transaction.

Installed layout

Path	Owner and mode	Purpose
`/etc/step-ca/config/ca.json`	`root:step 0440`	CA, policy, and encrypted provisioner configuration; no password field
`/etc/step-ca/certs/root_ca.crt`	`root:step 0440`	Public offline-root certificate
`/etc/step-ca/certs/intermediate_ca.crt`	`root:step 0440`	Online issuer certificate
`/etc/step-ca/secrets/intermediate_ca_key`	`root:step 0440`	Encrypted online issuer key
`/etc/step-ca/secrets/intermediate-password`	`root:root 0600`	Source for the systemd credential
`/var/lib/step-ca`	`step:step 0700`	Embedded database and writable state
`/etc/systemd/system/step-ca.service`	`root:root 0644`	Exact hardened service unit
`/usr/local/bin/verify-step-ca`	`root:root 0755`	Target-side acceptance verifier

The unit uses `ProtectSystem=strict`, an invisible process view, empty ambient and bounding capability sets, a read-only `/etc/step-ca`, a single writable state directory, systemd credentials, syscall and namespace restrictions, exact socket-bind permission, and default-deny IP filtering. `IPAddressAllow` contains loopback plus only the declared client networks. External firewalls remain mandatory because a service sandbox is not a network segmentation substitute.

Verify and collect acceptance evidence

Run the verifier after setup, package upgrades, configuration changes, restore tests, and network-policy changes:

```
sudo verify-step-ca \
    --server-name ca.internal.example.invalid \
    --bind-address 10.20.30.40 \
    --port 8443 \
    --default-tls-hours 24 \
    --max-tls-hours 720 \
    --allowed-client-network 10.20.0.0/16
```

The verifier requires root so it can prove that the encrypted intermediate key matches its certificate without exposing the password. It verifies:

- exact Step CLI and step-ca package versions;
- file ownership, modes, and single-link constraints;
- absence of a root private key anywhere below `/etc/step-ca`;
- exact configuration paths, lifetime claims, TLS floor, JWK-only provisioners, scoped X.509 policy, and absence of a plaintext password;
- exact systemd unit content and source-network policy;
- active/enabled service and a live main process;
- private DNS resolving only to the configured listener;
- exactly one listener on the CA port and no wildcard/alternate listener;
- intermediate chain and key pairing;
- HTTPS `/health` through the configured DNS identity and public root.

Acceptance evidence should also include tests from one allowed client and at least one denied source, issuance and renewal of a disposable certificate, application adoption, alert delivery, a backup restore rehearsal, time synchronization status, and a review of recent `step-ca` journal entries.

Backups and recovery

Back up the embedded database, public certificates, encrypted intermediate key, configuration, service unit, and the password through an approved secrets channel. Do not place the offline root key into an online-server backup. Encrypt backup media, separate backup credentials from host credentials, document retention, and test restoration on an isolated address.

For a single-node outage, restore to a patched compatible host, reinstate exact file ownership/modes, restore DNS and firewall policy, start the CA, run the verifier, and issue/renew a disposable certificate before returning production clients. A backup that has not passed a timed restore exercise is not accepted recovery evidence.

Intermediate rotation requires overlapping trust distribution: publish the new intermediate chain first, switch issuance only after relying parties trust it, retain the old intermediate until every certificate it issued has expired, and then remove it through a separately reviewed change. Root rotation is a formal ceremony and client-trust migration, not a normal package/configuration update.

Reproduce the PDF handoff

The Markdown file is canonical. With ReportLab installed, run the component and renderer tests, regenerate the review artifact after every accepted source change, then render every page to images and visually inspect it before promotion:

```
tests/smallstep-ca
PYTHONDONTWRITEBYTECODE=1 python3 tests/smallstep-ca-pdf.py
smallstep-ca/build-security-standard-pdf.py --check
smallstep-ca/build-security-standard-pdf.py
pdf_render_dir=$(mktemp -d "${TMPDIR:-/tmp}/smallstep-ca-pdf.XXXXXX")
pdftoppm -png -r 150 \
    output/pdf/smallstep-ca-security-standard.pdf \
    "$pdf_render_dir/page"
```

The generated PDF is a public client-neutral security handoff. Deployment evidence and private CA metadata require their own private as-built artifact.

References

- Smallstep: production considerations
- Smallstep: step-ca configuration
- Smallstep: certificate renewal
- Smallstep: certificate authority policies
- Step CLI 0.30.6 release
- step-ca 0.30.2 release